Introduction to Temporal Programming

23 Sep 2025, SumUp

Agenda

What “Temporal” means ?

Problems & traditional approaches

Modern approaches & state vs orchestration

Wrap-up

Temporal — from Latin *temporalis* (“of time”), from *tempus* (“time”): relating to time.

In programming theory (Lamport & Backus): reasoning about state and its evolution over time (temporal logic, state transitions, recurrence relations).

In distributed systems: managing **long-lived workflows and durable execution across real-world time** (fault tolerance, retries, timers, scheduling).

Real world examples

Flight booking – Hold a seat for 15 minutes, retry payment if it fails, release the seat when time runs out.

Banking\Ledgers – Settlement delays between accounts, reconciliation windows and timing constraints.

Payments (taxi) – User tries to pay, retries if it fails, can tip or adjust charges, and may change the payment method later.

Traditional Approaches



Cron Jobs (just runs the program periodically)



Database-Centric Approaches (Using relational tables to track process state, retries, and timers.)



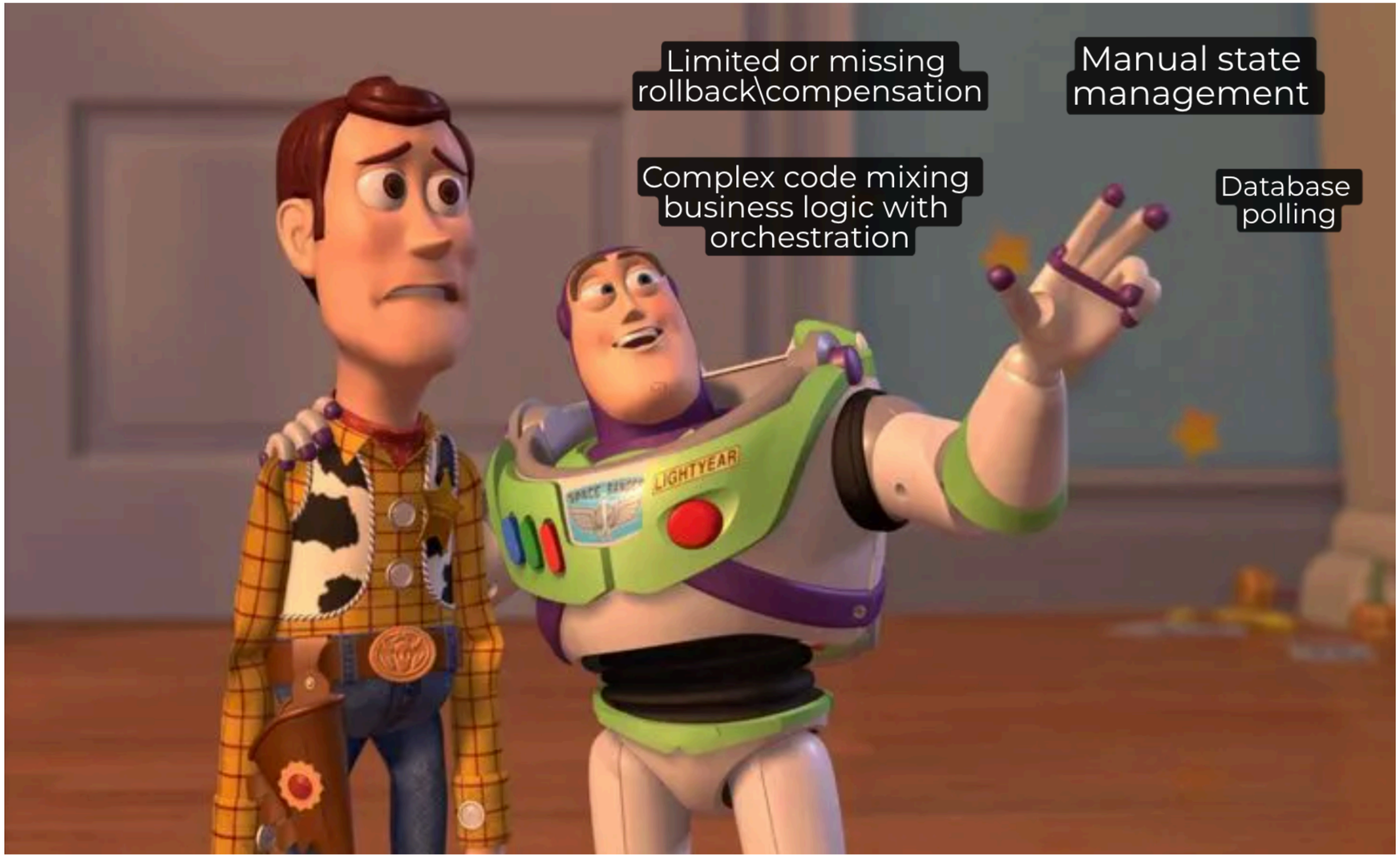
Message Queues / Job Queues Alone (no built-in timeout, retry, ordering, and state reconciliation)



Job schedulers (automatic timeout, retry, basic orchestration)

Are we handling temporal relationships right?

- Non-atomic workflows (sometimes “orchestrated” by UI app)
- Basic workflows composed of Oban jobs



Modern Approaches: Orchestration & Workflow Engines


Azure Durable Functions



AWS Step Functions


Google Cloud Workflows



Cadence


Temporal.io

Cadence \ temporal.io

- Building blocks: workflows and activities
- Each activity is durable (exactly once guarantees)
- Out of the box timeouts and retries
- Long delays

```go
// Send an email every 30 days, for the year
func SleepForDaysWorkflow(ctx workflow.Context) error {
  for i := range 12 {
    // Activities have built-in support for timeouts and retries!
    _ = workflow.ExecuteActivity(ctx, SendEmail).Get(ctx, nil)

    // Sleep for 30 days (yes, really)!
    _ = workflow.Sleep(ctx, time.Hour*24*30)
  }
}
```

Oban VS temporal.io (pseudocode)

```elixir
defmodule IssueCard do
  use Oban.Pro.Workers.Workflow, queue: :card_issuing, max_attempts: 14

  def process(%Job{args: %{"token" => token}} = job) do
    with {:ok, card} <- Card.get_by_token(token),
         {...} <- call_mpts_api(...) do
      trigger_jobs(response, job, card)  # ❌ Manual orchestration
    else
      {:error, :issue_failed} -> mark_as_rejected(job, token)
    end
  end

  defp trigger_jobs(%{response_type: :successful}, job, card) do
    job
    |> append_workflow()
    |> add(:set_limits, SetLimits.new(...), deps: [:issue_card])      # ❌ Manual deps
    |> add(:update_details, UpdateDetails.new(...), deps: [:issue_card])
    |> add(:enroll_3ds, Enroll3ds.new(...), deps: [:issue_card])
    |> add(:activate, Activate.new(...), deps: [:issue_card])
    |> add(:mark_active, UpdateStatus.new(...),
          deps: [:set_limits, :enroll_3ds, :activate])              # ❌ Complex deps
    |> OpentelemetryOban.insert_all()
  end
end

# ❌ Each job must reconstruct context from database
defmodule SetLimits do
  def process(%Job{args: %{"token" => token}}) do
    with {:ok, card} <- Card.get_by_token(token),  # ❌ Re-fetch everything
         {...} <- get_related_data(...) do
      call_mpts_set_limits(...)
    end
  end
end
```

```go
func CardIssuanceWorkflow(ctx workflow.Context, params CardParams) error {
    // Issue card
    var issueResult IssueResult
    err := workflow.ExecuteActivity(ctx, IssueCardActivity, params).Get(ctx, &issueResult)
    if err != nil {
        return workflow.ExecuteActivity(ctx, MarkRejectedActivity, ...).Get(ctx, nil)
    }

    // ✅ Automatic orchestration - no manual job coordination needed
    // ✅ Pass issueResult directly - no database reconstruction needed
    setLimitsFuture := workflow.ExecuteActivity(ctx, SetLimitsActivity, SetLimitsParams{
        Token: params.Token,
        CardDetails: issueResult.CardDetails,  // ✅ Direct data flow
    })
    updateDetailsFuture := workflow.ExecuteActivity(ctx, UpdateDetailsActivity, ...)
    enroll3DSFuture := workflow.ExecuteActivity(ctx, Enroll3DSActivity, ...)
    activateFuture := workflow.ExecuteActivity(ctx, ActivateActivity, ...)

    // ✅ Natural dependencies - just wait for what you need
    if err := setLimitsFuture.Get(ctx, nil); err != nil { return err }
    if err := updateDetailsFuture.Get(ctx, nil); err != nil { return err }
    if err := enroll3DSFuture.Get(ctx, nil); err != nil { return err }
    if err := activateFuture.Get(ctx, nil); err != nil { return err }

    // ✅ Simple coordination - no complex dependency graphs
    return workflow.ExecuteActivity(ctx, MarkActiveActivity, ...).Get(ctx, nil)
}

// ✅ Context received directly - no database reconstruction needed
func SetLimitsActivity(ctx context.Context, params SetLimitsParams) error {
    return mptsClient.SetLimits(params.CardDetails, ...)  // ✅ Data already available
}
```



Our services currently operate at the edge of what traditional state management systems can handle. In the future, adopting workflow orchestrators will better support complex, long-running, multi-step business processes.